

AHSANULLAH UNIVERSITY OF SCIENCE AND TECHNOLOGY



Department of Computer Science and Engineering

Course No: CSE 2104

Course Title: Data Structures Lab

Assignment No: 04

Date of Submission: 29/07/2026

Submitted by,

Name: Hasibul Hasan

Student ID: 00724205101098

Lab Section: B2

Academic Semester: Fall 2025

Program: BSc in CSE

CSE2104 Offline-4 Report

Tasks / Questions:

Implement the below functions of singly linked list in a single .cpp file.

- (1) insert first
- (2) insert last
- (3) insert anywhere between two nodes (by position, by value)
- (4) delete first
- (5) delete last
- (6) delete from anywhere between two nodes (by position, by value)
- (7) printing() -> function to print the linked list
- (8) searching() -> function to search a value in the linked list
- (9) last_node() -> function to print the value of the last node
- (10) previous_of_last_node() -> function to print the value of the previous node of last node
- (11) list_size() -> function to print the size of the linked list
- (12) reversePrint() -> function to print the linked list in reverse order
- (13) main function -> no manual node creation/connection is allowed

Source Code (C++):

```
#include<bits/stdc++.h>
using namespace std;
struct node {

    int value;
    node *next;

};
node *head=NULL;
void insertFirst(int n)
{

    node *newnode;
    newnode=(node*)malloc(sizeof(node));
    newnode->value=n;
    newnode->next=head;
    head=newnode;
}
void display()
{

    node *ghost;
    ghost=head;
    while(ghost!=NULL)
    {
        cout<<ghost->value<<endl;
        ghost=ghost->next;
    }
}
```

```

    }

}

void lastInsert(int x)
{
    node *king;
    node *newnode2;
    king=head;
    newnode2=(node*)malloc(sizeof(node));

    if(head==NULL)
    {
        head=newnode2;
        newnode2->value=x;
        newnode2->next=NULL;
        return;
    }

    while(king->next!=NULL)
    {
        king=king->next;
    }

    newnode2->value=x;
    king->next=newnode2;
    newnode2->next=NULL;
}

void anyByPosition(int p, int x)
{
    if(p == 0)
    {
        insertFirst(x);
        return;
    }

    node *newnode = (node*)malloc(sizeof(node));
    newnode->value = x;

    node *temp = head;

    for(int i = 1; i < p && temp != NULL; i++)
    {
        temp = temp->next;
    }

    if(temp == NULL)
    {
        cout << "Invalid Position!" << endl;
        free(newnode);
        return;
    }
}

```

```

        newnode->next = temp->next;
        temp->next = newnode;
    }

void anyByValue(int v, int x)
{

    node *king1 = head;
    node *king2 = head;

    node *newnode;
    newnode = (node*)malloc(sizeof(node));
    if(head->value == v)
    {
        insertFirst(x);
        return;
    }
    while(king1 != NULL && king1->value != v)
    {
        king1 = king1->next;
    }

    if(king1 == NULL)
    {
        cout << "Value not found!" << endl;
        return;
    }

    while(king2->next != king1)
    {
        king2 = king2->next;
    }

    newnode->value = x;
    newnode->next = king2->next;
    king2->next = newnode;
}

void deleteFront()
{
    if(head == NULL)
    {
        return;
    }

    node *temp = head;
    head = head->next;
    free(temp);
}

void deleteEnd()

```

```

{
    if(head == NULL)
    {
        cout << "List is empty!" << endl;
        return;
    }

    if(head->next == NULL)
    {
        free(head);
        head = NULL;
        return;
    }
    node *king1=head;
    node *king2=head;
    while(king1->next!=NULL)
    {
        king1=king1->next;
    }
    while(king2->next!=king1)
    {
        king2=king2->next;
    }
    king2->next=NULL;
    free(king1);
}
void searching(int x){
    node *king=head;
    if(head==NULL)
    {
        return;
    }
    while(king!=NULL && king->value!= x)
    {
        king=king->next;
    }
    cout<<" "<<endl;
    if(king!=NULL)
    {
        cout<<" Found"<<endl;
        return;
    }
    else
    cout<<" Not Found"<<endl;
}
void last_node()
{
    node* king=head;
    if(head==NULL)
    {
        cout<<"not found"<<endl;
    }
}

```

```

        return;
    }
    while(king->next!=NULL)
    {
        king=king->next;
    }
    cout<<"Last Node: "<<king->value<<endl;
}

void previous_of_last_node()
{
    node* t1=head;
    node* t2=head;
    if(head==NULL)
    {
        cout<<"Not Found"<<endl;
        return;
    }
    else if (head->next==NULL)
    {
        cout<<"Not Found"<<endl;
        return;
    }
    else{
        while(t1->next!=NULL)
        {
            t1=t1->next;
        }
        while(t2->next!=t1)
        {
            t2=t2->next;
        }
        cout<<"Before of Last Node: "<<t2->value<<endl;
    }
}

void list_size()
{
    if(head==NULL)
        return;
    int c=1;
    node* t=head;
    while(t->next!=NULL)
    {
        t=t->next;
        c++;
    }
    cout<<"size : "<<c<<endl;
}

void reverse()
{
    node *prev = NULL;

```

```

node *current = head;
node *next = NULL;

while(current != NULL)
{
    next = current->next;
    current->next = prev;
    prev = current;
    current = next;
}

head = prev;
}
int main()
{
    int n,m,x,l,p,v,z;
    cout<<"Insert at First "<<endl;
    cin>>n;
    for(int i=0;i<n;i++)
    {
        cin>>x;
        insertFirst(x);
    }
    cout<<"After Insertion at first : "<<endl;
    display();
    cout<<"Insert at Last "<<endl;
    cin>>m;
    for(int i=0;i<m;i++)
    {
        cin>>x;
        lastInsert(x);
    }
    cout<<"After Insertion at end : "<<endl;
    display();
    cout<<"insert anywhere between two nodes (by position) : ";
    cin>>p>>l;
    anyByPosition(p,l);
    cout<<"After insert anywhere between two nodes (by position) :
"<<endl;
    display();
    cout<<"insert anywhere between two nodes (by value) : ";
    cin>>v>>z;
    anyByValue(v,z);
    cout<<"After insert anywhere between two nodes (by value) :
"<<endl;
    display();
    cout << "After delete from front" << endl;
    deleteFront();
    display();
    cout << "After delete from end" << endl;
    deleteEnd();
}

```

```

        display();
    cout<<"search : ";
    cin>>x;
    searching(x);
    last_node();

    previous_of_last_node();
    list_size();
    cout << "Before Reverse:" << endl;
    display();
    reverse();
    cout << "After Reverse:" << endl;
    display();
}

```

Execution Trace: (User input is highlighted in *bold blue*)

Insert at First

2

1 6

After Insertion at first :

6

1

Insert at Last

2

5 6

After Insertion at end :

6

1

5

6

insert anywhere between two nodes (by position) : **2 100**

After insert anywhere between two nodes (by position) :

6

1

100

5

6

insert anywhere between two nodes (by value) : **1 500**

After insert anywhere between two nodes (by value) :

6

500

1

100

5

6

After delete from front

500

1

100

5

6

After delete from end

500

1

100

5

search : 5

Found

Last Node: 5

Before of Last Node: 100

size : 4

Before Reverse:

500

1

100

5

After Reverse:

5

100

1

500